



Final Examinations

2nd Semester, S.Y. 2024 – 2025
Transcribed by: Aerith Claude Catap
Instructor: Mr. Adrian Lee Magcalas

HAU School of Computing
First Year | Bachelor of Science in
Computer Science

Syllabus:
1. Classes and Objects 1.1. Creating a Class 1.2. Organizing Classes 1.3. Declaring Objects and Using their Methods 1.4. Constructor Method 1.5. Using Constructors 1.6. Overloading Constructors 1.7. Overloading a Method 1.8. Understanding Blocks and Scope 1.9. Using Automatically Imported, Prewritten Constants and Methods 1.10. Importing Classes 1.11. Understanding Composition and Nested Classes 2. Exceptions Handling 2.1. Exceptions 2.2. try Block 2.3. catch Block 2.4. throw Statement 2.5. finally Block 2.6. Throwing and Catching Multiple Exceptions 2.7. Specifying Exceptions in a Method 2.8. Unchecked vs Checked exceptions 2.9. Assertions 3. File Input/Output 3.1. Streams 3.2. Java File Class Methods 3.3. File Operations 3.4. Reading CSV Files in Java

**Should there be any mistakes, inconsistencies, or inaccuracies in this material, please feel free to reach out to the transcriber. Good luck and study well for your exams!*

Last updated: March 25, 2025 at 11:32 PM (1)

Classes
Classes ➤ Describes objects with common properties ➤ Act as blueprints for object creation. Attributes ➤ Characteristics that define an object ➤ Differentiate objects of the same class ➤ Store the object's state (values of its properties). Objects ➤ Specific, concrete instances of a class ➤ Every object is a member of a class ➤ "Is-a" Relationship – an object is a concrete example of its class. ○ Example: A Retriever is a Dog. ➤ Instantiation is the process of creating an object from a class. Example: Dog myRetriever = new Dog(); Methods ➤ A self-contained block of program code that carries out an action ➤ Similar to procedures or functions Class client or class user ➤ An application or a class that instantiates object of another prewritten class

Creating a Class
When creating a class, you must: ➤ Assign a meaningful class name. ➤ Determine attributes (data) and methods (behavior) ➤ Create a class header that consists of the following parts: 1. Access Modifier (optional) 2. The keyword <code>class</code> 3. Class name or identifier ➤ Example: <code>public class Employee</code> A class can have the following components: ➤ Data fields – variables declared within a class but outside of any method ➤ Instance variables – non-static fields given to each object of the class ➤ Private access for fields – fields should typically be private to enforce encapsulation. A class has the following principles: ➤ Encapsulation – keeping data hidden using private fields while providing access via public methods. Most methods are public, allowing interaction with object properties. ➤ Inheritance – classes can be extended using inheritance (<code>extends</code> keyword) A class may contain the following methods: 1. Mutator methods (Setters) ➤ Used to set or change the values of private fields. ➤ Ensure controlled modification of data. Example: <pre>public class Person { private String name; // Mutator method public void setName(String newName) { name = newName; } }</pre> 2. Accessor methods (Getters) ➤ Are used to retrieve values ➤ Allow controlled access to private attributes. ➤ Example: <pre>public class Person { private String name; // Accessor method public String getName() { return name; } }</pre> 3. Non-static methods (Instance Methods) ➤ Belong to objects, not the class itself. ➤ Require an instance of the class to be called, hence its name. Example: <pre>public class Car { private int speed; // Non-static method</pre>



Final Examinations

2nd Semester, S.Y. 2024 – 2025
Transcribed by: Aerith Claude Catap
Instructor: Mr. Adrian Lee Magcalas

HAU School of Computing
First Year | Bachelor of Science in
Computer Science

```
public void accelerate() {  
    speed += 10;  
}
```

Organizing Classes

- Place data fields in logical order
 - At the beginning of a class
 - List the fields vertically
- Data fields and methods may be placed in any order within a class
 - It's common to list all data fields first
 - Names and data types can be seen before reading the methods that use the data fields

Declaring Objects and Using their Methods

- Declaring a class does not create any actual objects
 - To create an instance of a class:
 - Supply a type and an identifier
 - Allocate computer memory for the object
 - Use the new operator
- Example:*
Employee someEmployee = new Employee();
- In the example above, the variable someEmployee does not store the actual object; it stores a memory address that points to the object.

Constructor Method

- A special method that creates and initializes objects.
- If you don't define a constructor, Java provides a default constructor.
- The constructor must have the same name as the class.

Example:

```
public class Employee {  
    String name;  
  
    // Constructor  
    public Employee(String empName) {  
        name = empName;  
    }  
}
```

So, when you create an Employee object:

```
Employee e1 = new Employee("Alice");
```

The variable e1 is initialized with the name "Alice."

Accessing Object Methods

- After an object is instantiated, its methods can be called using:
 - The object's identifier
 - A dot (.)
 - The method name

Example:

```
someEmployee.displayInfo();
```

Encapsulation and Data Hiding

- Encapsulation restricts direct access to data fields and enforces controlled access through methods.
- Private fields prevent direct modification from outside the class

- Public methods (get and set) control how data is accessed and modified.
- **set Method** – assigns values to private fields. It can include validation before setting the value.
- **get Method** – retrieves values from private fields. It provides read-only access if needed.
- It is important to note that the client application cannot access private fields directly. Fields are accessed using getter and setter methods.

Using Constructors

- **Constructors** initialize objects when they are created.
- The constructor name **matches the class name**.
- **Default constructors** require no arguments and are created automatically if no constructor is written.
- Default constructors initialize:
 - **Numeric fields** to 0
 - **Character fields** to '\u0000'
 - **Boolean fields** to false
 - **Object fields** to null
- Objects are instantiated using the new keyword.
- Example: Employee chauffeur = new Employee();

When declaring a constructor method, it:

- Must have the same name as the class it constructs
- Cannot have a return type
- A public access modifier

Creating and Calling Constructors with Parameter

- Java automatically provides a constructor method when class-created default constructors do not require parameters
- Writing your own constructor method:
 - Ensures that fields within classes are initialized to appropriate default values
 - Constructors can receive parameters
 - Used for initialization purposes
- When you write a constructor for a class, you no longer receive the automatically provided default constructor
- If a class's only constructor requires an argument, you must provide an argument for every object of the class

Overloading Constructors

- Overloading allows multiple constructors in the same class with different parameter lists.
- Java determines which constructor to use based on the number and types of arguments passed.
- Useful for providing different ways to initialize an object.

Example:

```
class Employee {  
    String name;  
    int age;  
  
    // Default constructor  
    public Employee() {  
        name = "Unknown";  
        age = 0;  
    }  
  
    // Constructor with one parameter  
    public Employee(String empName) {  
        name = empName;  
    }  
}
```



Final Examinations

2nd Semester, S.Y. 2024 – 2025
Transcribed by: Aerith Claude Catap
Instructor: Mr. Adrian Lee Magcalas

HAU School of Computing
First Year | Bachelor of Science in
Computer Science

```
        age = 0;
    }

    // Constructor with two parameters
    public Employee(String empName, int empAge) {
        name = empName;
        age = empAge;
    }
}

public class Main {
    public static void main(String[] args)
    {
        Employee emp1 = new Employee(); //
Calls default constructor
        Employee emp2 = new
Employee("Alice"); // Calls constructor
with one parameter
        Employee emp3 = new Employee("Bob",
30); // Calls constructor with two
parameters

        System.out.println(emp1.name + ", "
+ emp1.age); // Output: Unknown, 0
        System.out.println(emp2.name + ", "
+ emp2.age); // Output: Alice, 0
        System.out.println(emp3.name + ", "
+ emp3.age); // Output: Bob, 30
    }
}
```

Overloading a Method

- Method overloading is used when:
- Writing multiple methods with the same name but with different arguments
 - The compiler understands the meaning based on the arguments used with the method call
 - It is convenient for programmers to use one reasonable name for tasks that are functionally identical, but differ in argument types.

Example:

```
class MathOperations {
    // Method with one parameter
    int add(int a) {
        return a + 10;
    }

    // Method with two parameters
    int add(int a, int b) {
        return a + b;
    }

    // Method with three parameters
    int add(int a, int b, int c) {
        return a + b + c;
    }
}

public class Main {
    public static void main(String[] args)
    {
        MathOperations math = new
MathOperations();

        System.out.println(math.add(5));
// Output: 15
    }
}
```

```
        System.out.println(math.add(5,
10)); // Output: 15
        System.out.println(math.add(5, 10,
20)); // Output: 35
    }
}
```

Automatic Type Promotion in Method Calls

➤ If an

Method	Value That the Method Returns
abs(x)	Absolute value of x
acos(x)	Arc cosine of x
asin(x)	Arc sine of x
atan(x)	Arc tangent of x
atan2(x, y)	Theta component of the polar coordinate (r, theta) that corresponds to the Cartesian coordinate x, y
ceil(x)	Smallest integral value not less than x (ceiling)
cos(x)	Cosine of x
exp(x)	Exponent, where x is the base of the natural logarithms
floor(x)	Largest integral value not greater than x
log(x)	Natural logarithm of x
max(x, y)	Larger of x and y
min(x, y)	Smaller of x and y
pow(x, y)	x raised to the y power
random()	Random double number between 0.0 and 1.0
rint(x)	Closest integer to x (x is a double, and the return value is expressed as a double)
round(x)	Closest integer to x (where x is a float or double, and the return value is an int or long)
sin(x)	Sine of x
sqrt(x)	Square root of x
tan(x)	Tangent of x

Table 4-1: Common Math class methods

- application has only one version of a method, you must call it using a parameter that:
- Matches the method's required data type **or**
 - Can be automatically **promoted** to the correct data type.
- If a smaller data type is passed to a method expecting a larger data type, Java automatically promotes it in this order:
- int → long → float → double
- If the method expects double, you can pass an int, long, or float since they can be promoted.
- Java automatically expands smaller data types to fit larger expected types, avoiding errors.

Understanding Blocks and Scope

- Blocks**
- Use opening and closing curly braces
 - Can exist entirely within another block or entirely outside of and separate from another block
 - Cannot overlap
 - Types:
 - Outside block (or outer block)
 - Inside block (or inner block)
 - Nested (contained entirely within the outside block)
- Scope**
- The portion of a program within which you can refer to a variable.
 - Comes into Scope: A variable is created and becomes available for use.
 - Goes out of Scope: A variable ceases to exist when its block ends.
 - Redeclaring a Variable:
 - You **cannot** declare the same variable name more than once within the same block.
 - Attempting to do so results in an **error** (illegal action).



Final Examinations

2nd Semester, S.Y. 2024 – 2025
Transcribed by: Aerith Claude Catap
Instructor: Mr. Adrian Lee Magcalas

HAU School of Computing
First Year | Bachelor of Science in
Computer Science

- **Override** – occurs when you use the variable's name within the method in which it is declared. The variable takes precedence over any other variable with the same name in another method

Using Automatically Imported, Prewritten Constants and Methods

- Many classes are commonly used by a wide variety of programmers
- Package or library of classes
 - A folder that provides a convenient grouping for classes
 - Many contain classes available only if they are explicitly named within a program
 - Some classes are available automatically

java.lang package

- Implicitly imported into every Java program
- The only automatically imported, named package
- The classes it contains are fundamental classes (or basic classes)

java.lang.Math class

- Contains constants and methods used to perform common mathematical functions
- No need to create an instance
- Imported automatically
- Cannot instantiate objects of type Math, as the constructor for the Math class is private

Importing Classes

- **Using prewritten classes:** You can use Java's built-in classes or custom classes from external libraries.
- **Using the full class path:** Instead of importing, you can refer to a class by its full path (e.g., java.util.ArrayList list = new java.util.ArrayList();).
- **Importing a single class:** You can import a specific class using import java.util.ArrayList;.
- **Importing an entire package:** Use import java.util.*; to bring in all classes from the java.util package.
- **Wildcard (*) import:** Instead of specifying each class, you can use * to import all classes in a package.
 - There is no disadvantage in terms of performance when importing extra classes.
 - However, importing each class by name can be a form of documentation

Understanding Composition and Nested Classes

Composition

- Describes the relationship between classes when an object of one class data field is within another class
- Called a has-a relationship, because one class "has an" instance of another (e.g., a Car class has an Engine object).
- If the contained object does not have a default constructor, you must explicitly provide values for it when creating an instance of the outer class.

Example:

```
class Engine {
    private String type;

    // Constructor
    public Engine(String type) {
```

```
        this.type = type;
    }
}

class Car {
    private Engine engine; // Composition:
    Car "has an" Engine

    // Constructor
    public Car(String engineType) {
        this.engine = new
        Engine(engineType); // Providing a value
        for Engine
    }
}
```

Nested Classes

- A class within another class
- Stored together in one file
- Types:
 - static member classes
 - Nonstatic member classes
 - Local classes
 - Anonymous classes

Exceptions Handling

What are exceptions?

- Unexpected or error conditions
- Not usual occurrences within the program
- It may be caused by:
 - The program issues a call to a file that does not exist
 - The program attempts to write to a full disk
 - The user enters invalid data
 - The program attempts to divide a value by 0

Exception Handling

- **Exception handling** uses object-oriented techniques to manage errors gracefully.
- **Runtime exceptions** are a category of exceptions that occur during execution.
- Exceptions are **objects** that inherit from the **Throwable** class in Java.

Exceptions

Error Class

- Represents serious system-level failures that the program usually cannot recover from.
- Typically caused by critical issues like:
 - Running out of memory (OutOfMemoryError).
 - Failing to locate a required class (NoClassDefFoundError).

Exception Class

- Represents less severe errors that a program can handle and recover from.
- Occurs due to unexpected but manageable situations, such as:
 - Accessing an invalid array index (ArrayIndexOutOfBoundsException).
 - Performing illegal arithmetic operations, like division by zero (ArithmeticException).

Handling Exceptions in Java

- You do not always have to handle exceptions; the program may simply terminate.



Final Examinations

2nd Semester, S.Y. 2024 – 2025
Transcribed by: Aerith Claude Catap
Instructor: Mr. Adrian Lee Magcalas

HAU School of Computing
First Year | Bachelor of Science in
Computer Science

- However, abrupt termination is not ideal—it can lead to data loss or poor user experience.
- Instead of exception handling, you can use conditional statements (if-else) to prevent errors.
- Exception handling provides a more structured and elegant way to manage errors.
- Benefits of Exception handling:
 - Fault tolerance – The program continues running even when errors occur.
 - Robustness – The system remains resilient under unexpected conditions.

try Block

- A section of code where an **exception might occur**.
- If an error happens, Java looks for a way to handle it.
- Structure of a try block:
 - Starts with the keyword `try`
 - Enclosed in curly braces {}, which contains the statements that might cause an exception\
- If no exception occurs in the try block, the catch block does not execute.
- The program continues normally after the try block.
- The `getMessage()` method is used to retrieve details about the exception.
- When the try code fails:
 - It throws an exception
 - The `Exception` object is caught
 - The `catch` block executes
 - Control passes to statements at the end of the method

catch Block

- Follows the try block immediately.
- Handles exceptions thrown from the `try` block.
- Can catch exceptions of type `Exception` or its subclasses.
- A catch block includes:
 - The keyword `catch`
 - Opening and closing parentheses, containing:
 - An `Exception` type
 - A name for an instance of the `Exception` type
 - Opening and closing curly braces, containing:
 - Statements to handle the error condition
- Inside a `catch` block, you can:
 - Log the error for debugging.
 - Display an error message to the user.
 - Attempt to correct the issue instead of stopping the program.

throw Statement

- Used to manually trigger an exception.
- Sends an `Exception` object out of a block or method.
- Helps handle errors elsewhere in the program.

Example:

```
public class ThrowExample {
    static void checkAge(int age) {
        if (age < 18) {
            throw new
IllegalArgumentException("Age must be 18
or older.");
        }
    }
}
```

```
        System.out.println("Access
granted.");
    }

    public static void main(String[] args)
    {
        checkAge(16); // This will throw
an exception
    }
}
```

Output:
Exception in thread "main"
java.lang.IllegalArgumentException: Age
must be 18 or older.

finally block

- Runs regardless of whether an exception occurs or not.
- Ensures execution of critical code after a `try...catch` sequence.
- Used for cleanup tasks like closing files or releasing resources.
- The `try` block might throw an `Exception` object for which you did not provide a `catch` block
 - Program execution stops immediately
 - The exception is sent to the operating system for handling
 - The current method is abandoned
- The `finally` block executes no matter what outcome of the `try` block occurs, whether:
 - The `try` ends normally,
 - The `catch` executes, or
 - The exception causes the method to abandon prematurely

Throwing and Catching Multiple Exceptions

- You can place multiple statements within a `try` block. However, only the first error-generating statement throws an exception.
- A program can likewise have multiple `catch` statements. When a program contains multiple `catch` blocks:
 - They are examined in sequence until a match is found for the `Exception` type
 - The matching `catch` block executes
 - Each remaining `catch` block is bypassed
- **Catch-all block**
 - You can catch any type of exception using `catch(Exception e)`.
 - This is useful when you don't know the specific exception types in advance.
- When creating `try-catch` blocks, there may be certain parts of code that are unreachable:
 - Occurs when a statement can never be executed (e.g., code after a `return` or `throw` statement).
 - In exception handling, if a generic `catch(Exception e)` appears before specific exceptions, the specific `catch` blocks become unreachable.
- In Java 7 and 8, a `catch` block can also be written to catch multiple `Exception` types
- It is poor style for a method to throw more than three or four `Exception` types



Final Examinations

2nd Semester, S.Y. 2024 – 2025
Transcribed by: Aerith Claude Catap
Instructor: Mr. Adrian Lee Magcalas

HAU School of Computing
First Year | Bachelor of Science in
Computer Science

Specifying Exceptions in a Method

- If a method does not handle an exception but will pass it to another method, use the `throws` keyword in the method header.
- The exception specification lists all exceptions a method may throw.
- Every Java method has the potential to throw an exception, but:
 - For most cases, Java handles exceptions by terminating the program.
 - Only explicitly use `throws` when necessary to let the calling method handle the exception.
- It is good practice to catch and handle exceptions properly instead of always relying on `throws`.

Example:

```
import java.io.*;

public class Example {
    public static void readFile(String
fileName) throws IOException {
        FileReader file = new
FileReader(fileName); // May throw
IOException
        BufferedReader reader = new
BufferedReader(file);

        System.out.println(reader.readLine());
        reader.close();
    }

    public static void main(String[] args)
{
        try {
            readFile("test.txt");
        } catch (IOException e) {
            System.out.println("Error
reading file: " + e.getMessage());
        }
    }
}
```

In the example above,

- `readFile()` declares that it may throw an `IOException`, so it uses `throws IOException` in the method header.
- The `main()` method calls `readFile()` inside a `try` block and catches any `IOException` that occurs.
- If the file is missing or unreadable, the exception is caught, and an error message is displayed instead of crashing the program.

Unchecked vs Checked exceptions

Unchecked exceptions

- Inherit from the `Error` class or the `RuntimeException` class.
- Not required to be handled explicitly.
- If they occur, the program may terminate unexpectedly.
- Example: Dividing by zero (`ArithmeticException`).

Checked exceptions

- Must be handled using `try-catch` or declared using `throws`.
- Programmers should anticipate these exceptions.
- The program should recover from them instead of crashing.

- Example: Attempting to open a non-existent file (`IOException`).
- If you throw a checked exception from a method, you must do one of the following:
 - Catch it
 - Declare the exception in the method header's `throws` clause
- To use a method to its full potential, you must know:
 - Method's name
 - Method's return type
 - Type and number of arguments the method requires
 - Type and number of exceptions the method throws

Creating Your Own Exception in Java

- Java provides over 40 built-in exception categories, but you can create custom exceptions.
- Custom exceptions must extend a subclass of `Throwable`, usually `Exception` or `RuntimeException`.
- Exception class constructors:

```
Exception()
Exception(String message)
Exception(String message,Throwable cause)
Exception(Throwable cause)
```

Assertions

An **assertion** is:

- A Java feature used for detecting logic errors and debugging.
- Helps ensure that the program behaves as expected
- Assertions are disabled by default.
- To enable, run the program with the `-ea` flag:
- Syntax:

```
assert booleanExpression :
optionalErrorMessage;
```

- If the `booleanExpression` evaluates to `false`, Java throws an `AssertionError`.

Example:

```
public class AssertionExample {
    public static void main(String[] args)
{
        int age = -5;
        assert age >= 0 : "Age cannot be
negative!";
        System.out.println("Age is: " +
age);
    }
}
```

- If assertions are enabled, the program throws an `AssertionError` due to age being negative.
- If assertions are disabled, the program runs normally.

Points to remember:

- Don't forget that all the statements in a `try` block might not execute
- Don't forget you might need a `nextLine()` method call after an attempt to read numeric data from the keyboard throws an exception
- Don't forget that a variable declared in a `try` block goes out of scope at the end of the block



Final Examinations

2nd Semester, S.Y. 2024 – 2025
Transcribed by: Aerith Claude Catap
Instructor: Mr. Adrian Lee Magcalas

HAU School of Computing
First Year | Bachelor of Science in
Computer Science

- Don't forget that when a variables get its usable value within a `try` block, you must ensure that it has a valid value before attempting to use it
- Don't forget to place more specific `catch` blocks before more general ones
- Don't forget to write a `throws` clause for a method that throws an exception but does not handle it
- Don't forget to handle any checked exception thrown to your method

File Input/Output (I/O)

- Java I/O stream is the flow of data that you can either read from, or you can write to.
- It is used to perform read and write operations in file permanently.
- Java uses streams to perform these tasks. Java I/O stream is also called File Handling, or File I/O. It is available in `java.io` package.
 - `Java.io` package provides classes for system input and output through files, network streams, memory buffers, etc.

Streams

- Streams are the sequence of bits (data).
- There are two types of streams:
 - Input Streams – are used to read the data from various input devices like keyboard, file, network, etc.
 - Output Streams – are used to write the data to various output devices like monitor, file, network, etc.

Streams based on data

1. **Byte Stream** – used to read or write **byte** data.
 - a. **Byte Input Stream**
 - These are used to read byte data from various input devices.
 - `InputStream` is an abstract class and it is the super class of all the input byte streams.
 - b. **Byte Output Stream**
 - These are used to write byte data to various output devices.
 - `OutputStream` is an abstract class and it is the superclass for all the output byte streams.
2. **Character Stream** -- used to read or write **character** data
 - a. **Character Input Stream**
 - These are used to read `char` data from various input devices.
 - `Reader` is an abstract class and is the super class for all the character input streams.
 - b. **Character Output Stream**
 - These are used to write `char` data to various output devices.
 - `Writer` is an abstract class and is the super class of all the character output streams.

Java File Class Methods

Method	Return Type	Description
--------	-------------	-------------

<code>canRead()</code>	Boolean	Used to check whether we can read the data of the file or not.
<code>createNewFile()</code>	Boolean	Used to create a new empty file.
<code>canWrite()</code>	Boolean	Used to check whether we can write the data into the file or not.
<code>exists()</code>	Boolean	Used to check whether the specified file is present or not.
<code>delete()</code>	Boolean	Used to delete a file.
<code>getName()</code>	String	Used to find the file name.
<code>getAbsolutePath()</code>	String	Used to get the absolute pathname of the file.
<code>length()</code>	Long	Used to get the size of the file in bytes.
<code>list()</code>	String[]	Used to get an array of the files available in the directory.
<code>Mkdir()</code>	Boolean	Used for creating a new directory

File Operations

Create a File

- Create a File operation is performed to create a new file. We use the `createNewFile()` method of file.
- The `createNewFile()` method returns `true` when it successfully creates a new file and returns `false` when the file already exists.

```
// Importing File class
import java.io.File;
// Importing the IOException class for handling errors
import java.io.IOException;
class CreateFile {
    public static void main(String args[]) {
        try {
            // Creating an object of a file
            File f0 = new File("D:FileOperationExample.txt");
            if (f0.createNewFile()) {
                System.out.println("File " + f0.getName() + " is created successfully.");
            } else {
                System.out.println("File is already exist in the directory.");
            }
        } catch (IOException exception) {
            System.out.println("An unexpected error is occurred.");
            exception.printStackTrace();
        }
    }
}
```



Final Examinations

2nd Semester, S.Y. 2024 – 2025
Transcribed by: Aerith Claude Catap
Instructor: Mr. Adrian Lee Magcalas

HAU School of Computing
First Year | Bachelor of Science in
Computer Science

```
}
```

Get File Information

- The operation is performed to get the file information.
- We use several methods to get the information about the file like name, absolute path, is readable, is writable and length.

```
// Import the File class
import java.io.File;

class FileInfo {
    public static void main(String[] args)
    {
        // Creating a file object
        File file = new
File("D:FileOperationExample.txt");

        if (file.exists()) {
            // Displaying file information
            System.out.println("File Name:
" + file.getName());
            System.out.println("Absolute
Path: " + file.getAbsolutePath());
            System.out.println("Writable:
" + file.canWrite());
            System.out.println("Readable:
" + file.canRead());
            System.out.println("File Size
(bytes): " + file.length());
        } else {
            System.out.println("The file
does not exist.");
        }
    }
}
```

Write to a File

- In order to write data into a file, we will use the `FileWriter` class and its `write()` method together.
- We need to close the stream using the `close()` method to retrieve the allocated resources.

```
// Importing required classes
import java.io.FileWriter;
import java.io.IOException;

class WriteToFile {
    public static void main(String[] args)
    {
        try {
            // Creating a FileWriter object
            FileWriter writer = new
FileWriter("D:FileOperationExample.txt");

            // Writing content to the file
            writer.write("A File is a named
location used to store related
information.");

            // Closing the FileWriter
            writer.close();

            System.out.println("Content
successfully written to the file.");
        } catch (IOException e) {
            System.out.println("An
unexpected error occurred.");
            e.printStackTrace();
        }
    }
}
```

```
}
}
```

Read from a File

- In order to write data into a file, we will use the `Scanner` class.
- Here, we need to close the stream using the `close()` method. We will create an instance of the `Scanner` class and use the `hasNextLine()` method `nextLine()` method to get data from the file.

```
// Importing required classes
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

class ReadFromFile {
    public static void main(String[] args)
    {
        try {
            // Creating a File object
            File file = new
File("D:FileOperationExample.txt");
            Scanner dataReader = new
Scanner(file);

            // Reading and printing file
content
            while
(dataReader.hasNextLine()) {
                String fileData =
dataReader.nextLine();

                System.out.println(fileData);
            }

            // Closing the scanner
            dataReader.close();
        } catch (FileNotFoundException e) {
            System.out.println("Unexpected
error occurred!");
            e.printStackTrace();
        }
    }
}
```

Delete a File

- In order to delete a file, we will use the `delete()` method of the file.
- We don't need to close the stream using the `close()` method because for deleting a file, we neither use the `FileWriter` class nor the `Scanner` class.

```
// Importing the File class
import java.io.File;

class DeleteFile {
    public static void main(String[] args)
    {
        // Creating a File object
        File file = new
File("D:FileOperationExample.txt");

        // Attempting to delete the file
        if (file.delete()) {

            System.out.println(file.getName() + " was
deleted successfully.");
        } else {

```




Final Examinations

2nd Semester, S.Y. 2024 – 2025
Transcribed by: Aerith Claude Catap
Instructor: Mr. Adrian Lee Magcalas

HAU School of Computing
First Year | Bachelor of Science in
Computer Science

```
System.out.println("Error:
File deletion failed.");
    }
}
```

Reading CSV Files in Java

- The CSV stands for Comma-Separated Values.
- It is a simple file format which is used to store tabular data in simple text form, such as a spreadsheet or database.
- The CSV file used a delimiter to identify and separate different data token in a file. The default separator of a CSV file is a comma (,).
- There are following ways to print an array in Java:
 - Java Scanner class
 - Java String.split() method
 - Using OpenCSV API

Java Scanner class

- Java Scanner class provide various methods by which we can read CSV file.
- The Scanner class provides a constructor that produces values scanned from the specified file. It breaks data into the token form.
- It uses a delimiter pattern which by default matches' white space. The resulting tokens then converted into values of different types using the next() methods.

```
import java.io.File;
import java.util.Scanner;

public class ReadCSVExample {
    public static void main(String[] args)
throws Exception {
        // Parsing a CSV file into the
Scanner class constructor
        Scanner sc = new Scanner(new
File("F:\\CSVDemo.csv"));
        sc.useDelimiter(","); // Setting
the delimiter pattern

        // Reading and printing the CSV file
content
        while (sc.hasNext()) {
            System.out.print(sc.next() + "
"); // Displaying each token
        }

        sc.close(); // Closing the scanner
    }
}
```

Using OpenCSV API

- OpenCSV is a third party API which provide standard libraries to read various versions of CSV file. The library provides better control to handle the CSV file. The library can also read TDF (Tab-Delimited File) file format.
- Features of OpenCSV:
 - Any number of values per line.
 - Ignores commas in quoted elements.
 - Handles entries that span multiple lines.
- The CSVReader class is used to read a CSV file. The class provides CSVReader class constructor to parse a CSV file.
- Syntax:

```
public CSVReader(Reader reader, char
separator)
```

OR

```
public CSVReader(Reader reader)
```

- Parameters:
 - reader – The reader to a CSV source.
 - separator – It is a delimiter which is used for separating entries.

Example:

```
import java.io.FileReader;
import com.opencsv.CSVReader;
import java.io.IOException;

public class ReadCSVExample3 {
    public static void main(String[] args)
    {
        try (CSVReader reader = new
CSVReader(new
FileReader("F:\\CSVDemo.csv"))) {
            String[] nextLine;

            // Reads one line at a time
            while ((nextLine =
reader.readNext()) != null) {
                for (String token :
nextLine) {
                    System.out.print(token
+ " ");
                }
                System.out.println();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```